



Research Collaborators Status Updates

Rushil - Kalshi Trading

Previous Problem:

- PAI was not implemented correctly onto baseline model, ending up performing worse
- Fixed the PAI implementation to load the trained baseline and only add capacity on top which guarantees it can never perform worse

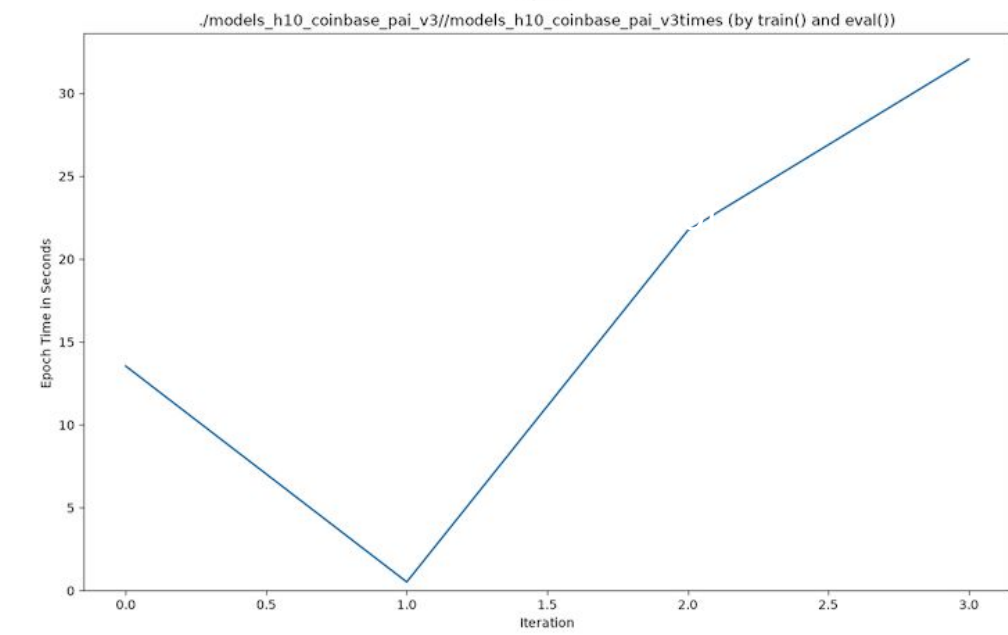
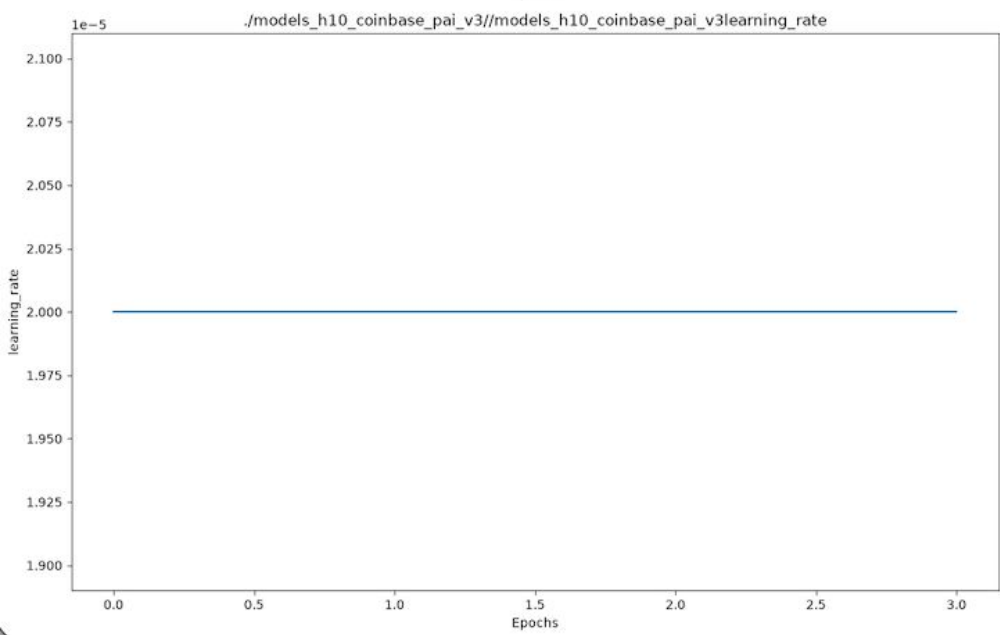
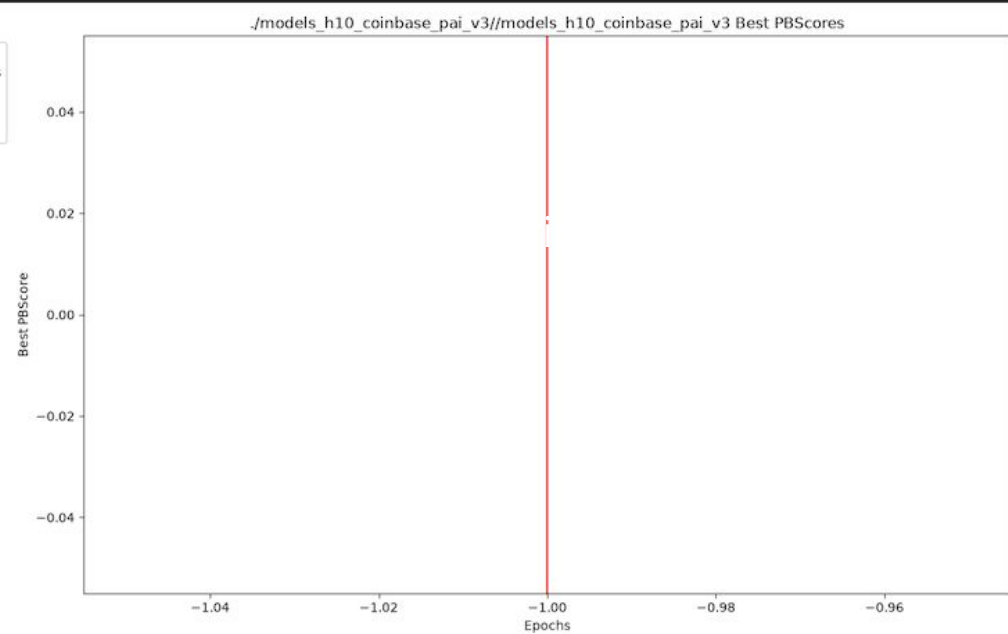
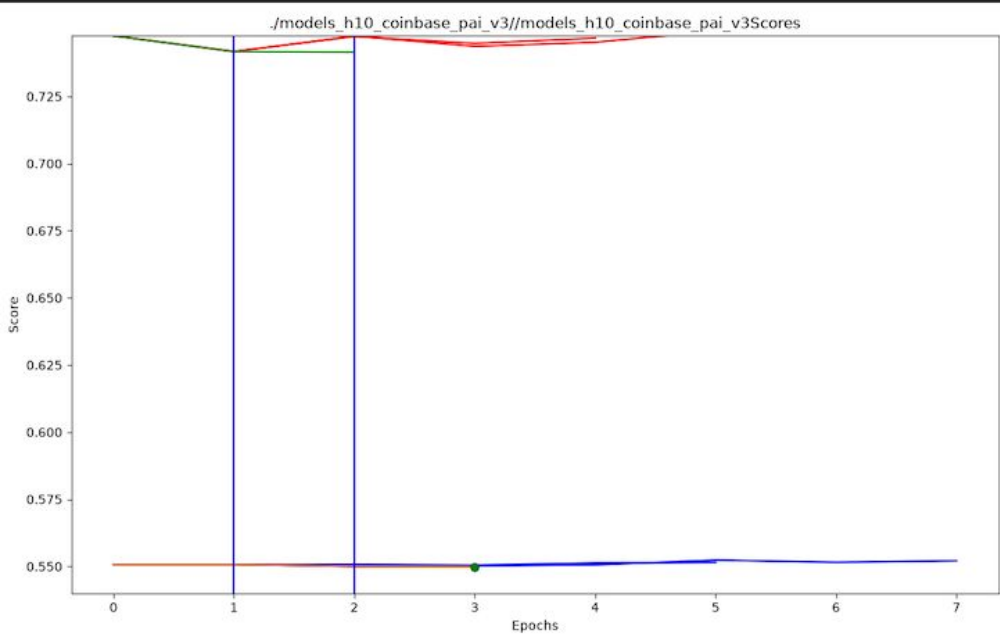
What I worked on:

- Reduced features on baseline model (12/29 were considered noise)
- Pooled 5 more crypto markets in addition to BTC (ETH, SOL, XRP, BNB, DOGE) to train on, increased profits by 60.6%
- Ran PAI testing on top of this new baseline, which didn't end up contributing much

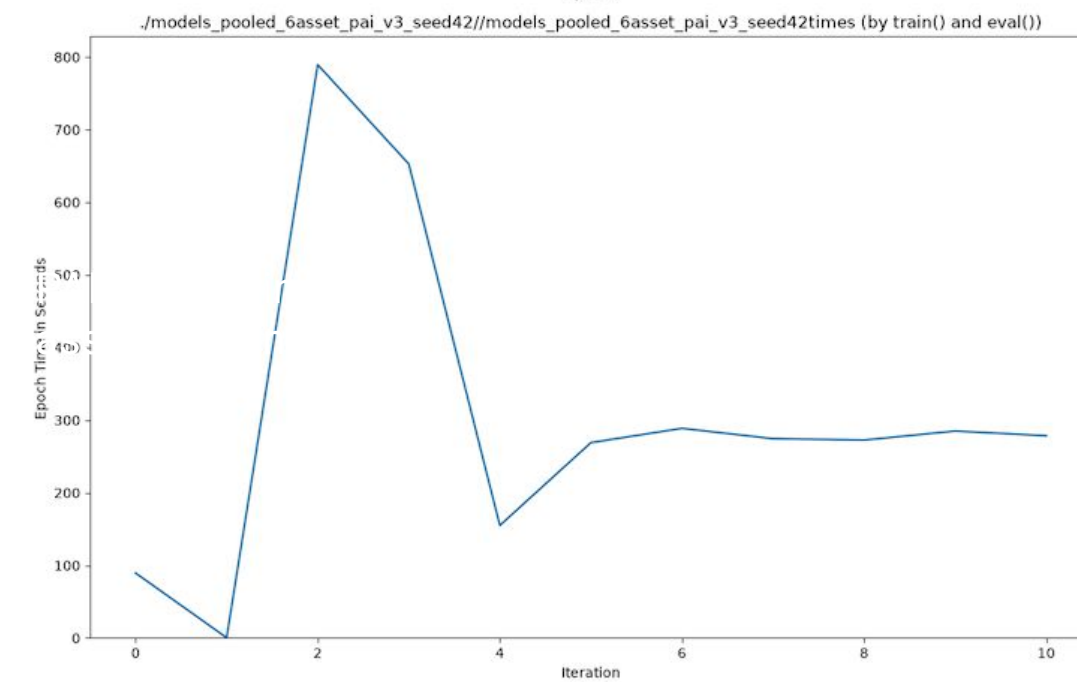
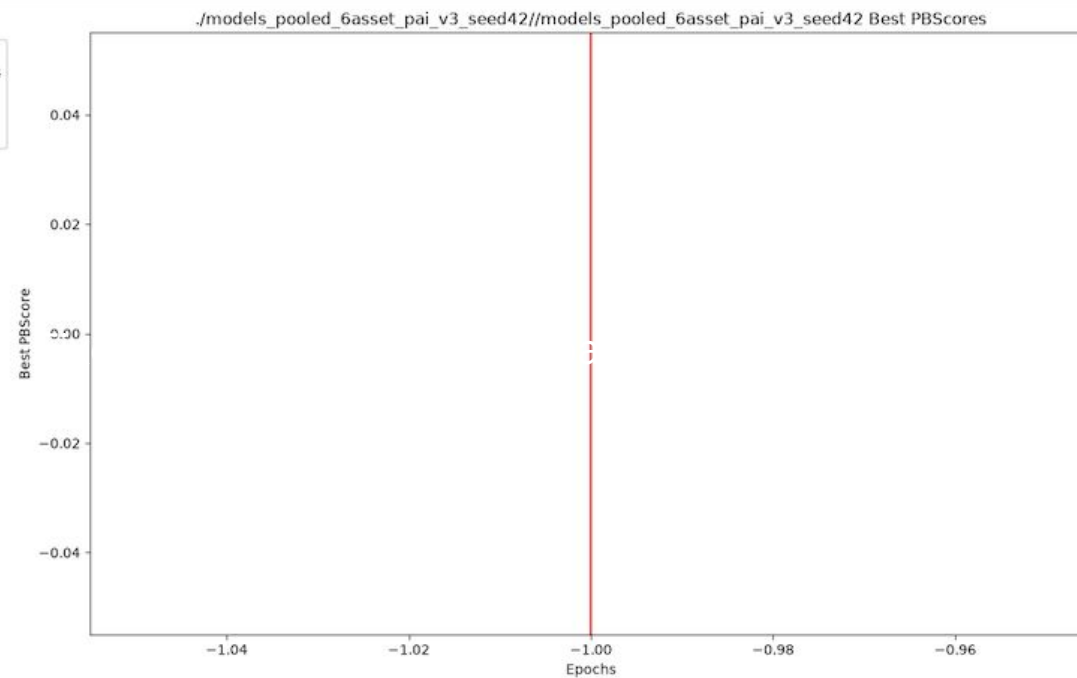
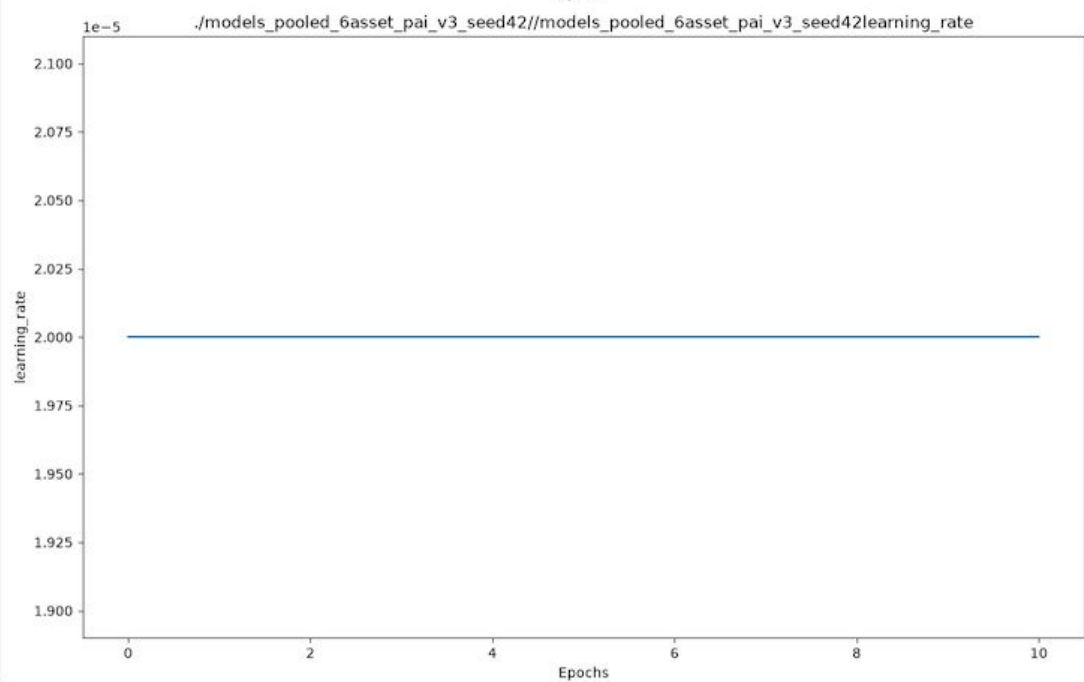
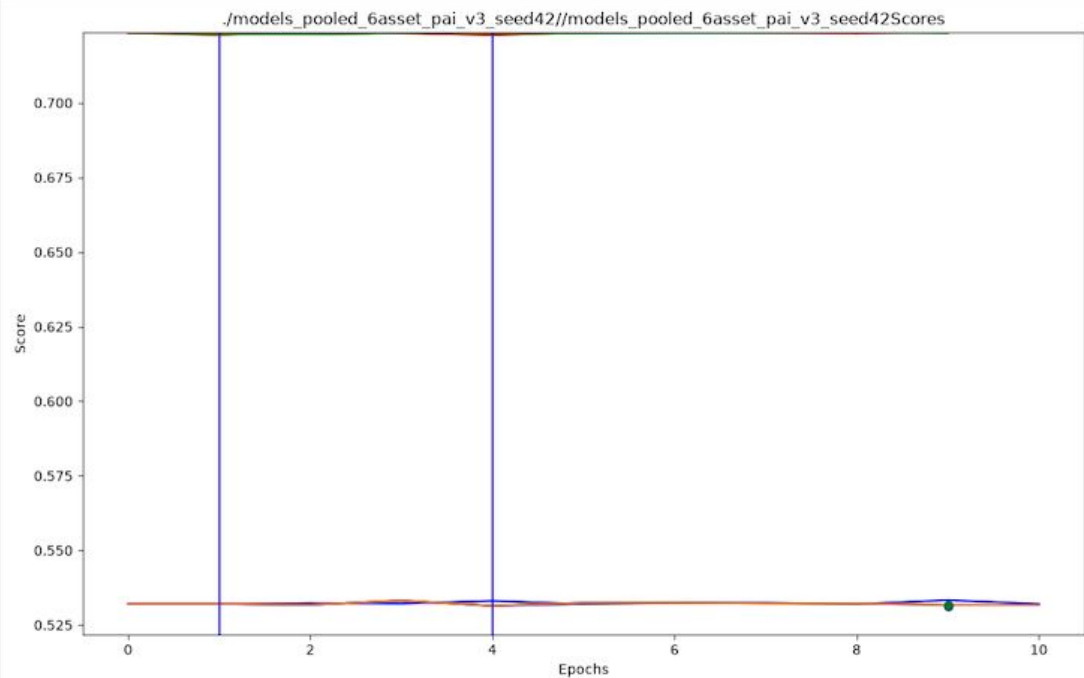
Exploration:

- Read Perforated Backpropagation paper
- In the PAI Examples repo, there is a project on a Vision Transformer similar to mine (PyTorch attention) which had a 4% improvement
- Possible to gain Perforated Backpropagation license for testing?

Rushil - Kalshi Trading



Run



Lakshmi - structured pruning

Proposed 3 Starting Methods

L1-norm filter pruning — prunes filters with the smallest L1 norm; simple, widely cited, good baseline level check

Network Slimming — batch-norm scaling factors as an importance signal, prunes channels below a threshold

Taylor/gradient-based pruning — ranks filters by estimated impact on the loss using a first-order Taylor expansion

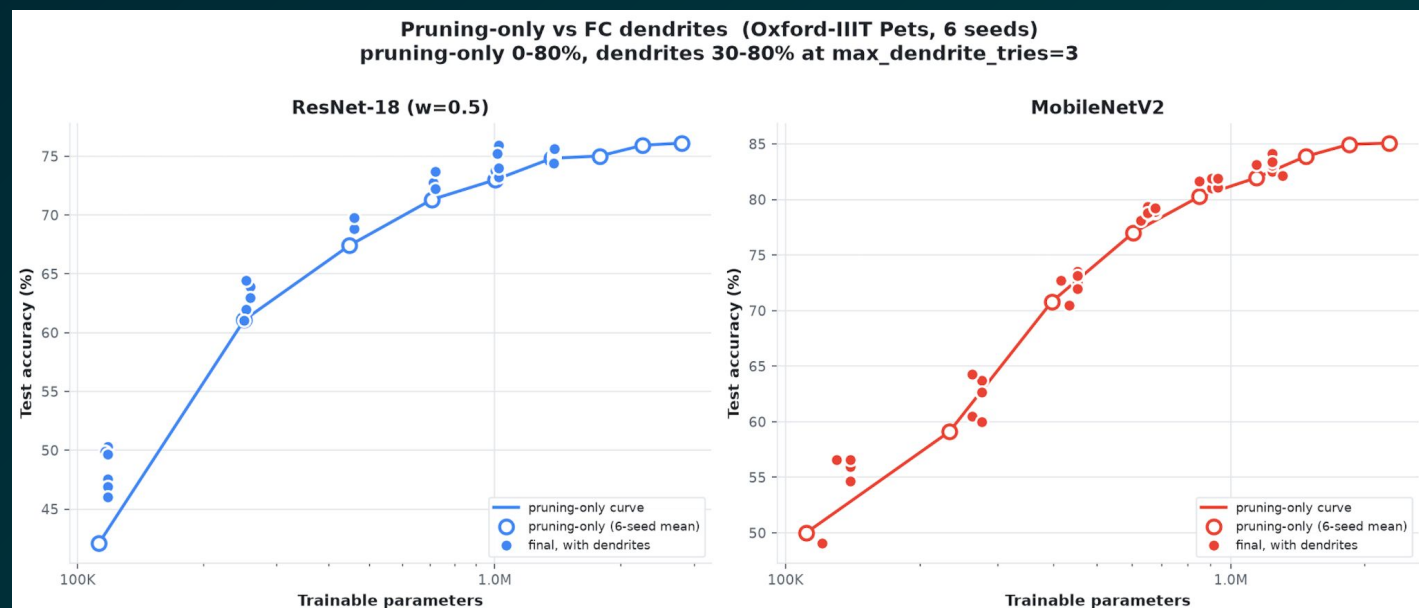
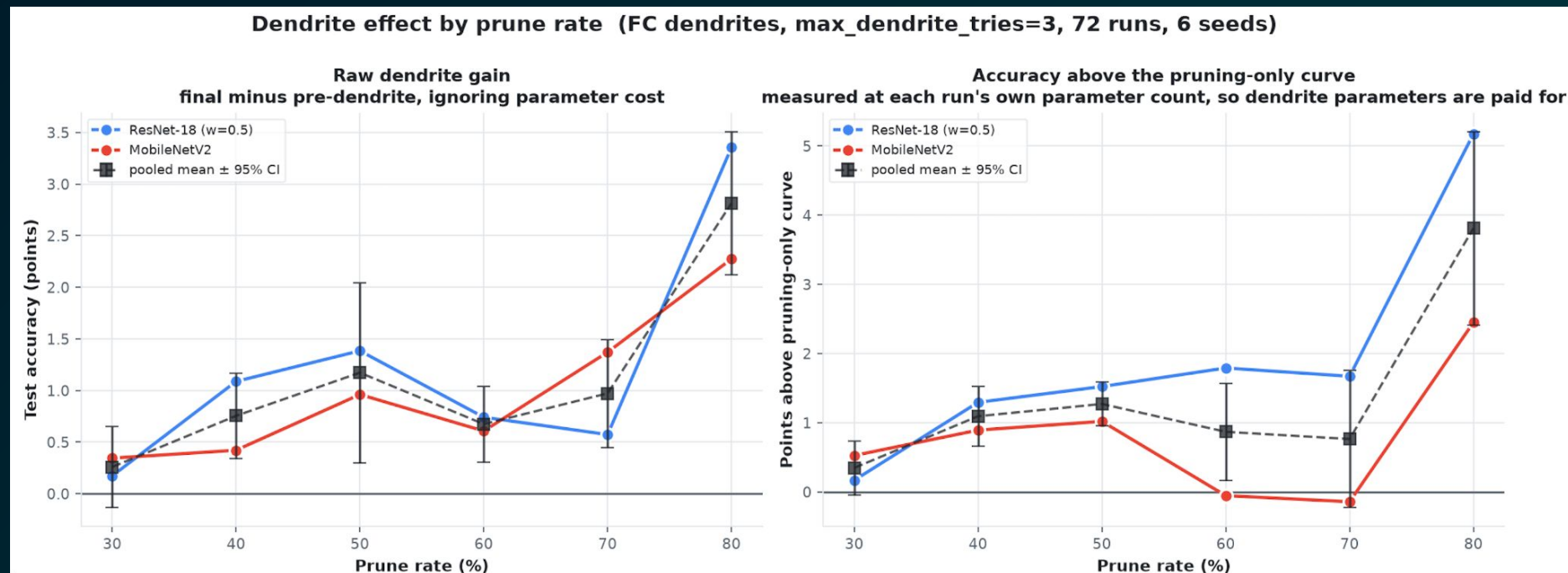
next steps —

- implement/replicate baseline methods on ResNet
- standard benchmark (CIFAR-10 or ImageNet subset) to replicate numbers

Vigya - Pets Perforation

Progress:

- Full re-run at uniform settings. 6 seeds, prune rates 0 to 80%, licensed PB package, max_dendrite_tries=3 everywhere. 72 perforation runs on top of the pruning-only study.
- Patience-only stopping, no epoch cap anywhere. Every run in both arms terminates on patience 8 at 0.1% minimum improvement, so nothing is cut short and nothing overruns.
- Backfilled 40/60/80 at seeds 0 to 2, which had originally run at max_dendrite_tries=1. Nothing now pools across two different settings.
- Extended the pruning-only study to 9 rates so the curve is properly resolved in the region where the dendrite results actually land.

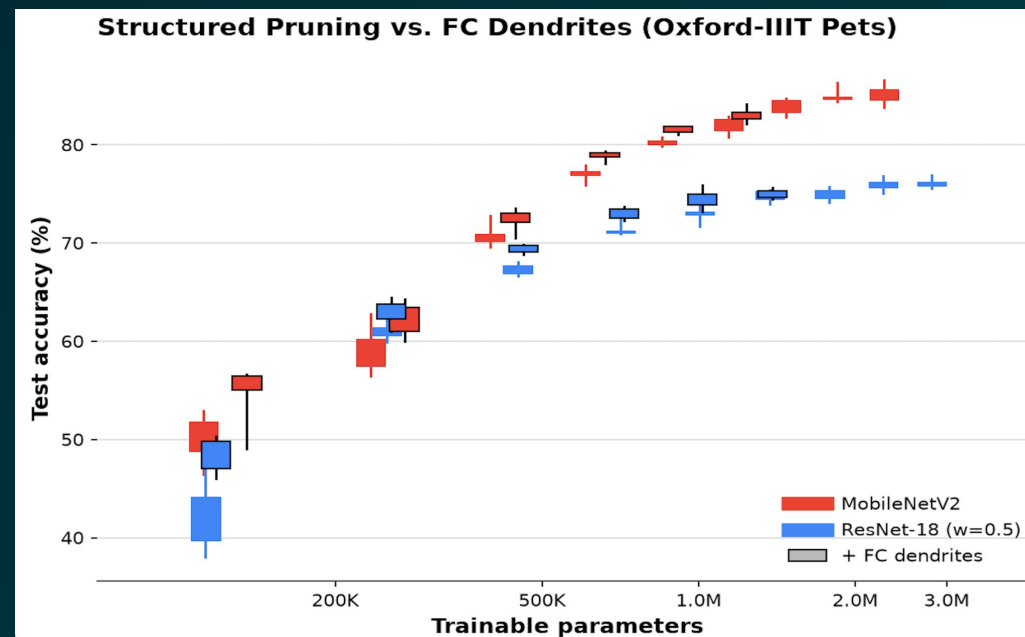
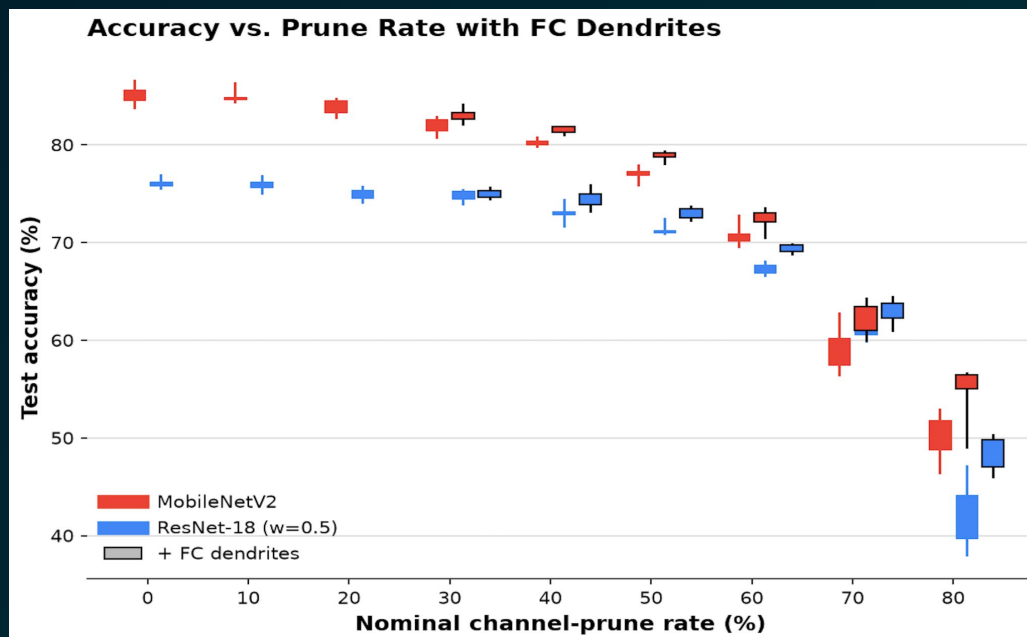


Vigya - Pets Perforation Run

Results:

- Across all 72 runs, perforated models land **+1.36 points above the pruning-only curve**, 95% CI [0.94, 1.78], $p = 1.1\text{e-}08$, 59 of 72 runs positive.
- The effect scales with how hard you prune. Near zero at 30%, around +1 point from 40 to 60%, and +3.81 at 80%.
- 80% is the clean signal. All 12 seeds gain raw accuracy there, +2.82 points, $p = 7\text{e-}06$.
- It splits by architecture. ResNet-18 is +5.18 above the curve on 6 of 6 seeds. MobileNetV2 is +2.45 on 5 of 6, and goes flat at 60 and 70%.

prune rate	ResNet-18 (w=0.5)	MobileNetV2
30%	+0.17 (4/6 positive)	+0.53 (4/6)
40%	+1.29 (6/6)	+0.90 (6/6)
50%	+1.52 (6/6)	+1.02 (6/6)
60%	+1.79 (6/6)	-0.05 (3/6)
70%	+1.67 (5/6)	-0.14 (2/6)
80%	+5.18 (6/6)	+2.45 (5/6)



Dendrite Counts and Their Cost

Results:

- PAI settles on 2 to 3 dendrites nearly everywhere. Of 72 runs, 49 end with three, 12 with two, 6 with one, 4 with none.
- Dendrite count carries no signal about prune rate. Mean is between 2.2 and 3.0 in every single cell, both models, every rate.
- What differs is price. ResNet pays 1,924 to 6,660 parameters per dendrite. MobileNet pays 9,472 to 33,189.
- That 5x ratio is classifier width. MobileNetV2's head takes 1280 dimensions, ResNet-18 at w=0.5 takes 256. FC dendrites scale with it.

ResNet-18 (w=0.5)

prune	0	1	2	3	mean	parameters per dendrite	total +params
30%	0	0	2	4	2.7	6,660	1.3%
40%	1	0	1	4	2.3	5,698	1.3%
50%	0	1	0	5	2.7	4,773	1.8%
60%	0	0	0	6	3.0	3,811	2.5%
70%	1	2	0	3	1.8	2,849	2.1%
80%	0	0	1	5	2.8	1,924	4.8%

MobileNetV2

prune	0	1	2	3	mean	parameters per dendrite	total +params
30%	1	0	0	4 (+1 at k=5)	2.8	33,189	8.2%
40%	1	0	2	3	2.2	28,453	7.3%
50%	0	1	2	3	2.3	23,717	9.2%
60%	0	1	1	4	2.5	18,981	12.0%
70%	0	0	2	4	2.7	14,245	16.3%
80%	0	1	1	4	2.5	9,472	21.3%

Pooled that's 4 runs at zero, 6 at one, 12 at two, 49 at three, and one anomalous run at five.

Next steps:Add prune rates 10, 20 and 90 to the perforation sweep

Leo and Vishy - Quantization

Vishy: Currently doing the second training run where we train dendritic models until `training_complete` is True.

Leo: Writing a setup shell script and run scripts for AWS training runs.

Next steps: Rerun the first training run version to fix previous perforated errors.

Rodela - Perforated Brain Decoder (Intro)

Two phase project plan:

- (Phase 1) Replicate Défossez 2022 speech-decoding baseline on the gwilliams2022 MEG dataset
 - paper achieves 41% top-1 accuracy identifying which sentence a subject heard from 1,300+ candidates)
- (Phase 1a) Smoke test. Prove the entire pipeline runs end-to-end on a small subset (dset.n_recordings=4)
- (Phase 2) Perforate the ~6.4M-parameter brain module (dilated ConvNet)

The architecture we're replicating

```
Audio stimulus (what subject heard) → Wav2Vec 2.0 (FROZEN) → target embeddings
                                                                    |
                                                                    | contrastive
loss
                                                                    ▼
MEG signals from subject → Brain module (~6.4M params, TRAINABLE) → predicted
embeddings
```

Only the brain module is trained. Wav2Vec 2.0 is a frozen pretrained encoder. Phase 1 is to reproduce this pipeline; Phase 2 will replace the brain module's training procedure with Perforated Backpropagation.

References:

Gwilliams2022 dataset:

<https://www.nature.com/articles/s41467-022-34326-1>

brainmagick repo:

<https://github.com/facebookresearch/brainmagick>

OSF Data Hosting: <https://osf.io/ag3kj/overview>

Dora experiment framework:

<https://github.com/facebookresearch/dora>

Paper (Nature MI 2023):

<https://www.nature.com/articles/s42256-023-00714-5>

Rodela - Perforated Brain Decoder (07/14-07/21)

Setup:

- Downloaded to local device and attempted to train.
Colab cannot have full gwilliams2022 download (~112 GB projected vs. ~52 GB free at peak)
- **Smoke Test passed:** `dora run dset.selections=[gwilliams2022] dset.n_recordings=4`
 - Data loading works
 - Preprocessing runs. Bandpass filter (0.5–30 Hz) + downsample (1000 → 120 Hz)
 - Model instantiates, Forward pass works
 - Contrastive loss computes (between brain embeddings and Wav2Vec 2.0)
 - Backward pass works, loss decreases across epochs (sanity check), checkpoints save
- Training on going on RunPod (watchdog set as claude code)

Query: Baseline trainable parameter count depends on where we draw the boundary: 6.4M for the dilated ConvNet encoder alone (paper's Table S1), 7.67M including the projection head, 9.57M including projection head + subject embeddings. Which one do we use as our baseline for the Phase 2 delta? I'd default to the 9.57M (full trainable footprint of what actually gets trained end-to-end) unless we match the paper's 6.4M convention for easier external comparison.

Rodela (Background, 7/21-7/30)

- Dataset: Gwilliams2022 (MEG-MASC)
 - 27 native English speakers MEG brain recording (~2GB/session) while listening to 4 fictional stories
 - 2 identical 1hr sessions on separate days
 - 56 hours of recording total across 208 MEG sensors || 112 GB full study

```
gwilliams2022/download/  
├─ participants.tsv          # subject list  
├─ sub-01/ ... sub-27/  
│   ├─ ses-0/meg/           # session 0  
│   │   ├─ *_task-0_meg.con  # raw MEG, one file per story  
│   │   ├─ *_events.tsv      # word/phoneme onset timings  
│   │   ├─ *_channels.tsv    # sensor metadata  
│   │   └─ *_markers.mrk     # head position markers  
│   └─ ses-1/meg/           # session 1  
└─ stimuli/  
    ├─ audio/                # the .wav files subjects heard  
    └─ text/                  # transcripts
```

- Model's task: Retrieval. Hence uses contrastive loss (unlike cross entropy or any other).
- Given a 3-second window of MEG, identify which speech segment the subject was hearing, from a pool of 1,300+ candidate segments not seen during training

Rodela (Preprocessing Detail)

- On the MEG side:
 - Load raw .con files via MNE-Python
 - Bandpass filter, 0.5–30 Hz (removes slow drift below 0.5 Hz and high-frequency muscle/line noise above 30 Hz); frequency band kept where speech-tracking neural activity lives
 - Downsample 1000 Hz to 120 Hz — 30 Hz is the highest frequency we care about
 - Robust scaling per channel: MEG sensors have wildly different baseline amplitudes; normalizing prevents a few loud channels from dominating
 - Clamping of extreme outlier values (artifacts from eye blinks, jaw movement, etc.)
- On the audio side:
 - Load the .wav stimulus files
 - Wav2Vec 2.0 forward pass (frozen) → target embeddings.
- Alignment:
- Event-based time alignment: events.tsv files record when each word/phoneme was spoken, which is how MEG windows get matched to the correct audio embedding

Rodela (Model Detail):

model: clip_conv config in brainmagick maps to a model called SimpleConv (custom)

Stage	What it does	Why it's there
Subject embedding layer	Learned per-subject matrix applied to input	Every brain is anatomically different; MEG sensor N doesn't measure the same thing across people. This lets the model adapt per subject.
Spatial attention over sensors	Attention across the 208 MEG channels	Learns which sensors matter, using sensor 2D layout positions
Initial 1×1 conv	Channel projection	Maps into the model's working dimension
Dilated residual conv blocks	Stack of 1D convolutions with increasing dilation, residual connections	This is the "brain module" core. Dilation widens the receptive field exponentially without adding parameters – needed because speech-relevant context spans hundreds of milliseconds.
Projection head	Final linear layer(s)	Maps into Wav2Vec 2.0's embedding dimension so contrastive loss can compare them

- **6.4M = dilated conv blocks only (paper Table S1)**
- **7.67M = + projection head**
- **9.57M = + subject embeddings**

Rodela

Preprocessing		
	Paper	My run
Resample target	120 Hz (Torchaudio)	120 Hz
Highpass	not stated	0 (none)
Lowpass	not stated	0 (none)
Bandpass applied	not stated	No
Baseline correction	subtract each channel's mean over first 0.5 s	0.5 s pre-onset, MNE <code>baseline=(None, 0)</code>
Scaler	sklearn robust scaler, [-1,1] ↔ [0.25, 0.75] quantile	RobustScaler, 0.25/0.75, per recording
Clamping	20 SD	20 SD (<code>max_scale: 20.0</code>), <code>clip: true</code>)
Window ("sample")	3 s	3.0 s (tmin -0.5, tmax 2.5)
Brain→audio offset	+150 ms	+150 ms
Speech model	wav2vec2-large-xlsr-53	same
Speech target layers	mean of last 4 transformer layers	layers 14–18 (5 layers) ⚠
Split ratios (train/valid/test)	70 / 20 / 10	70 / 10 / 20 ⚠
Split unit	by segment; same segment always same split across repetitions; no identical sentences across splits; overlapping segments excluded	by segment (block uid = SHA-256 of word sequence, deterministic threshold)
Valid shares subjects with train	yes (implied)	yes (all 27 subjects in all splits)

Training		
	Paper	My run
Optimizer	Adam	Adam
Learning rate	3e-4	3e-4
Batch size	256	256
Loss	CLIP / InfoNCE, in-batch negatives	<code>clip</code>
Epoch definition	1,200 updates (fixed)	794 updates (loader-limited; <code>max_batches: 1200</code> never binds)
Examples per epoch	307,200	203,152
Valid batches per epoch	not reported	138
Epoch cap	not reported	200 (never reached)
Early stopping	patience 10 on valid	patience 10
Model selection	best valid loss	best valid loss
Eval frequency	every epoch (implied)	<code>eval_every: 1</code>
Random inits	3	3 (seeds 2036 / 2037 / 2038)
Training hardware	2 × V100 16 GB	1 × Apple Silicon GPU (MPS)
world_size	2	1
Eval hardware	not stated	A100 40 GB (Modal)
Wall-clock per run	not reported	11.49 h / 13.24 h / 11.50 h
Epochs to best valid	not reported	4 (= 3,176 updates)
Epochs to early stop	not reported	14

Rodela

Results — accuracy

	Paper (Gwilliams MEG)	My run (mean ± SD, 3 seeds)	Δ
Top-10 segment	70.7 ± 0.1 %	70.294 ± 0.597 %	−0.41 pp
Top-5 segment	not reported	62.230 ± 0.652 %	—
Top-1 segment	41.3 ± 0.1 %	40.869 ± 0.564 %	−0.43 pp

Seed	Signature	Top-1	Top-5	Top-10
2036	97d170e1	40.220 %	61.477 %	69.608 %
2037	47ff097d	41.239 %	62.595 %	70.695 %
2038	d28aee4a	41.148 %	62.618 %	70.580 %

Retrieval Accuracy: How well the model ranks the correct answer among candidates.

The setup: the model sees 3 seconds of MEG. It computes a similarity score between that brain representation and every candidate speech segment in the test set (1,428). Those scores get sorted, highest first (Top-1).

The paper led with top-10 segment.

	Paper	My run (3-seed mean)
Best valid loss	not reported	2.0359
Train loss at best	not reported	1.4727
Generalization gap at best	not reported	+0.563
Final valid loss (ep. 14)	not reported	2.5889
Final train loss (ep. 14)	not reported	0.5444
Generalization gap at final	not reported	+2.044
Gap growth factor	not reported	3.6×
Valid-loss rise after best	not reported	+0.553

Seed	Best epoch	Best valid	Train @ best	Gap @ best	Final valid	Gap @ final
2036	4	2.0380	1.5033	+0.535	2.5085	+1.966
2037	4	2.0272	1.4548	+0.572	2.5913	+2.045
2038	4	2.0425	1.4601	+0.582	2.6668	+2.123
Mean	4	2.0359	1.4727	+0.563	2.5889	+2.044

Rodela

Per-epoch, 3-seed mean

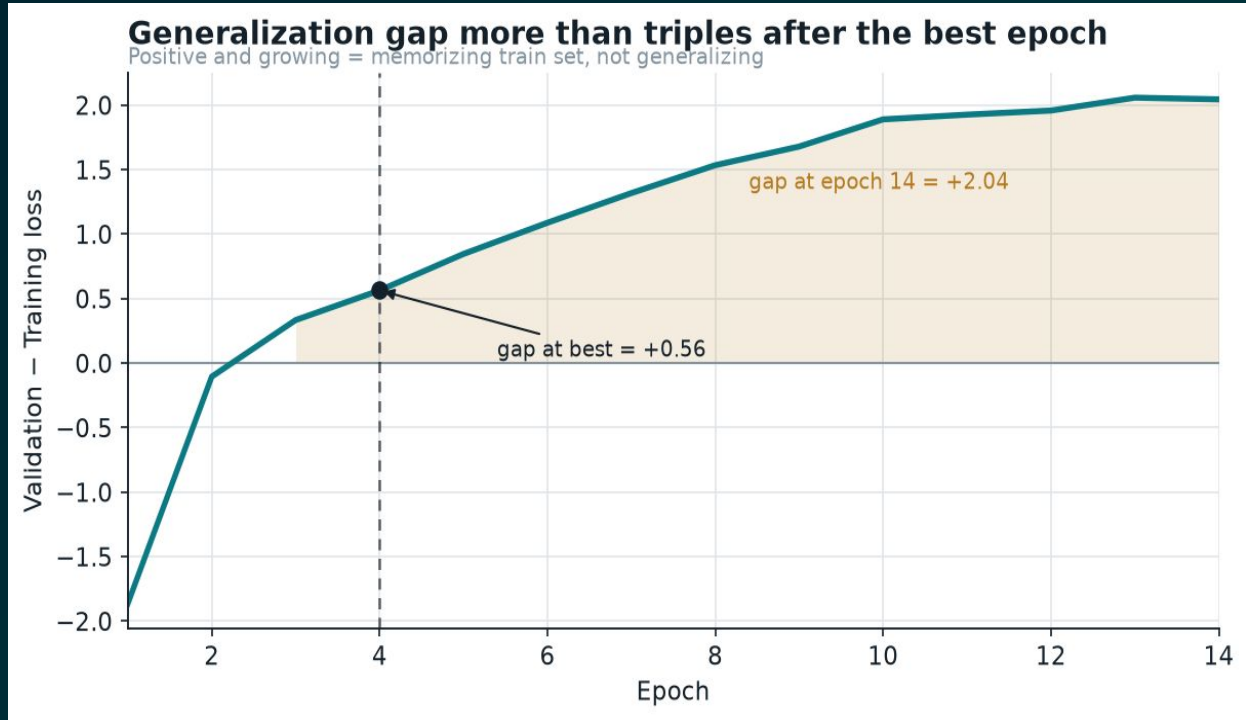
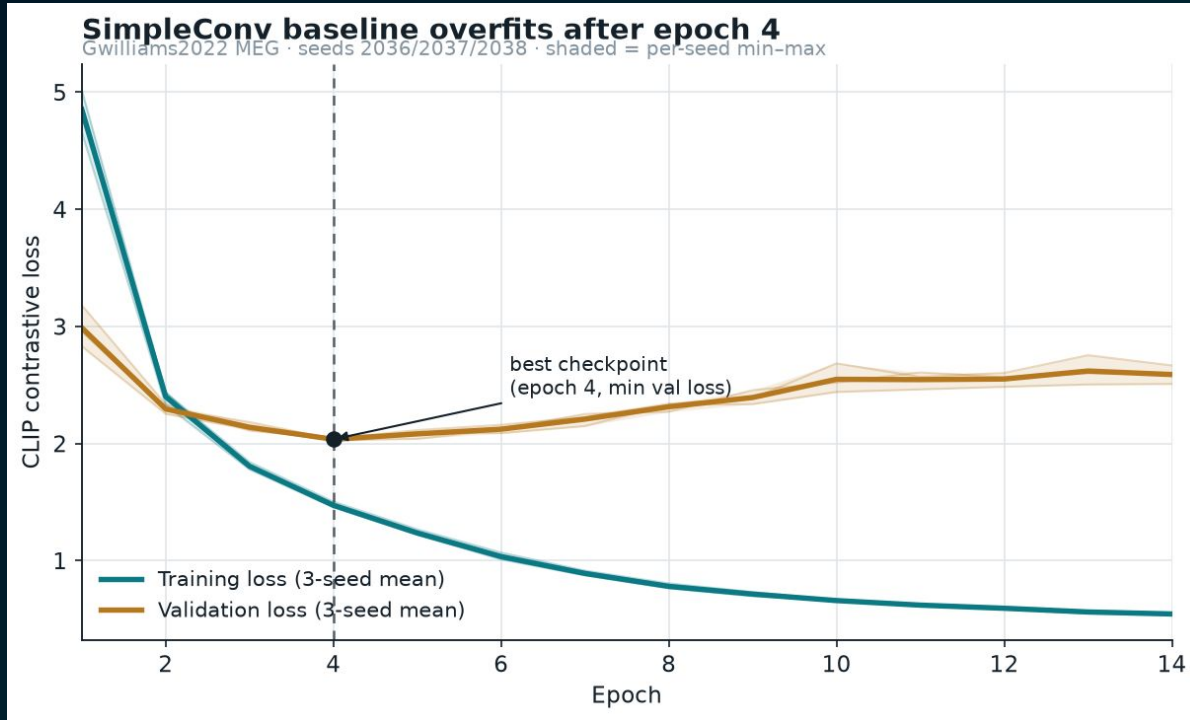
Epoch	Updates (cum.)	Train (mean)	Valid (mean)	Gap	Valid 2036	Valid 2037	Valid 2038
1	794	4.8514	2.9878	-1.8637	2.9529	2.8295	3.1809
2	1,588	2.4000	2.2948	-0.1052	2.2961	2.2531	2.3352
3	2,382	1.8056	2.1390	+0.3334	2.1831	2.1244	2.1097
4	3,176	1.4727	2.0359	+0.5632	2.0380	2.0272	2.0425
5	3,970	1.2364	2.0823	+0.8458	2.0927	2.0394	2.1147
6	4,764	1.0345	2.1223	+1.0878	2.0880	2.1191	2.1598
7	5,558	0.8909	2.2084	+1.3175	2.1489	2.2520	2.2242
8	6,352	0.7802	2.3140	+1.5337	2.3308	2.2711	2.3400
9	7,146	0.7143	2.3928	+1.6785	2.3363	2.4555	2.3866
10	7,940	0.6589	2.5481	+1.8892	2.4398	2.5205	2.6841
11	8,734	0.6199	2.5457	+1.9258	2.4619	2.6071	2.5681
12	9,528	0.5931	2.5507	+1.9576	2.4840	2.5653	2.6028
13	10,322	0.5615	2.6190	+2.0575	2.5039	2.5990	2.7541
14	11,116	0.5444	2.5889	+2.0445	2.5085	2.5913	2.6668

Seed summary

Seed	Signature	Best epoch	Best valid	Train @ best	Gap @ best	Final epoch	Final valid	Final train	Gap @ final	Valid rise after best
2036	97d170e1	4	2.0380	1.5033	+0.5347	14	2.5085	0.5426	+1.9659	+0.4705
2037	47ff097d	4	2.0272	1.4548	+0.5724	14	2.5913	0.5468	+2.0445	+0.5641
2038	d28aee4a	4	2.0425	1.4601	+0.5824	14	2.6668	0.5438	+2.1230	+0.6243
Mean		4	2.0359	1.4727	+0.5632	14	2.5889	0.5444	+2.0445	+0.5530

- Across all seeds, the replicated SimpleConv reaches best validation loss after ~3,176 updates/(4 full passes over the data), then validation loss climbs while training loss keeps falling.
- The train/validation gap more than triples (from +0.56 to +2.04). This is paper's own early-stopping operating point
- Training loss never plateaus. It falls monotonically from 4.85 to 0.54, still dropping at epoch 14. The model has plenty of capacity left to fit training data but it just stopped converting that into validation performance after four passes.

Rodela



Model has plenty of capacity left to fit training data but it stopped converting that into validation performance after four passes.

Represents specific shape of generalization-limited, not capacity-limited. So, adding dendrites may need a design decision

Rodela- Questions for next step

- What counts as success for the "new signal domain" push, beating the paper's accuracy, matching it with fewer parameters, or just showing Perforated Backpropagation (PB) trains
- Does PB tend to help or hurt when the base is already overfitting? Do dendrites carry any implicit regularization, or is a smaller base required for a fair test?
 - A: Perforate the baseline as-is (Does PB improve the exact replication paper model, in practice)
 - B: Regularize first, then perforate (Does PB help once full-size base isn't overfitting)
 - C: Shrink the base, perforate to recover (recover a larger model's accuracy from a deliberately smaller / underfit SimpleConv with fewer or comparable parameters- claude recommended)
- For a ~6M-param 1-D conv contrastive model, what default is recommended: max_dendrites, history vs fixed switch mode, switch cadence (let me know if this doesn't make sense). Also 30 epochs would means ~23,800 updates on our setup (so should we switch every 30 s, which is 794 updates)
- How should I checkpoint a restructured model and reload it in a separate CUDA process, so the official segment evaluator can score it? (This is my current blocker)
- Perforate the encoder Conv1d (current scope), or the subject-adaptation path (given the split is by-segment and PB's BCI pitch was per-subject calibration)?

Rodela (Plan for next steps)

HELD FIXED ACROSS EVERY CELL — SO COMPARISONS ARE VALID

Gwilliams2022 data + by-segment 70/10/20 split

Preprocessing / cache

Adam, LR 3e-4

Batch 256

CLIP contrastive loss

Best-checkpoint-by-valid-loss

Official run_eval_probs on CUDA

Seeds 2036 / 2037 / 2038

THE THREE FACTORS WE VARY		
Base size, regularization, perforation		
FACTOR 1 — BASE ARCHITECTURE		
ID	CHANGE	WHY
B0	full: width 320, depth 10	Replicated baseline — 9.57M, top-10 70.3% (reference)
B1	width 320→256	~35% smaller encoder; mild underfit, likely recoverable
B2	width 320→192	~65% smaller; strong underfit, bigger efficiency headline if recovered
B3	depth 10→6	Fewer blocks; receptive field vs channel capacity
FACTOR 2 — REGULARIZATION		
ID	CHANGE	WHY
R0	none (merger_dropout 0.2 only)	Reference regime — overfits after ~4 data passes
R1	+ weight decay 1e-4	Cheapest, broad; no architecture change
R2	+ conv dropout 0.1	Regularize encoder capacity directly
R3	+ input dropout 0.1	Sensor / spatial robustness
R4	merger dropout 0.2→0.3	Regularize the spatial-attention merger
Rwd+cd	R1 + R2	Combination you suggested
Ra1l	R1 + R2 + R3 + R4	Full stack — the "all regularization" combination
FACTOR 3 — PERFORATION		
ID	SETTING	WHY
P0	none	Per-cell un-perforated reference
P1	history mode, max_dendrites 3, n_epochs_to_switch 3	Your defaults, tuned for our ~14-epoch runs (default 10 too long)
P2	forced switch every 2 epochs (also try 1, 3)	Your overfitting suggestion

WHY NOT THE FULL GRID

4 bases × 7 regs × 3 PB × 3 seeds ≈ 252 runs

At ~12 h/run on a single Apple-MPS Mac that is ~125 days of compute — and beyond the "keep it to the basics first" guidance. Instead we use a **screening funnel**: cheap single-seed screens judged on validation-loss (local, no CUDA eval, since overfitting shows by ~epoch 6) narrow the field; only finalists get 3 seeds + CUDA eval. Every base and every regularizer is still run at least once — nothing is silently dropped.

A

Regularization screen

Base B0, seed 2036, no PB, short runs. Screen all 7 regularizers on validation-loss gap. → the 1–2 that best curb overfitting.

B

Base-size screen

Bases B1/B2/B3, seed 2036, no PB. Metric: validation loss + parameter count. → how much each smaller base underfits, and the gap perforation must close.

C

Perforation on finalists

Best (base × reg) cells with P1 (then P2 if promising), **3 seeds**, CUDA-eval for top-1/5/10. Headline: accuracy vs total params vs 9.57M.

The MEG baseline overfits, so the target is matching paper’s accuracy with fewer parameters. Experiment matrix, screening strategy outlined below

GRANULARITY OPTIONS (SINGLE-SEED SCREENING RUNS)		
DESIGN	SCREEN RUNS	ASSUMES
Full base × reg cross	28	nothing — most thorough
Reg screen on B0 + each smaller base at 2 reg levels (none / B0-winner)	~13	catches the regxsize interaction cheaply
Funnel as first drafted	~10	best reg transfers across sizes (weak)
Trimmed funnel	~6	plus fewer regularizers

Rodela (Update on Phase 2, 08/04)

- PAI run works.
 - Perforated the 15 encoder Conv1d (6.10M of the 9.57M model), tracked everything else so the treatment stays confined to the encoder.
 - Verified that the correct scope (15 perforated, heads excluded), dendrites attach and grow (3 rounds), n/p switching over 7 epochs, loadable best_model_pai.pt
- Had a roadblock. **Learning:** PB requires every optimizer parameter to be perforated or tracked.
- Running a small capacity test:
 - 4 recordings / 1 batch per epoch, testing_dendrite_capacity mode inflates dendrites on purpose (so the ~18M dendrite params are a stress-test maximum, not the efficiency result), tiny base, and flat-noise validation loss. No accuracy numbers.
- **Next steps:**
 - One full perforated run on the baseline (testing_dendrite_capacity=False, full data), dendrites grow dynamically; measure params added and where.
 - Load that grown model via load_pai_model and score it on CUDA with the official evaluator (first top-1 / top-10)
 - The sweep (smaller base + regularization + perforation *****targeting ~70% for top-10 segment under 9.57M params*****)